Chapter ML:IV

IV. Neural Networks

- Perceptron Learning
- Gradient Descent
- Multilayer Perceptron
- Radial Basis Functions

©STEIN 2005-2020

Gradient Descent

Gradient descent is a first-order iterative optimization algorithm for finding a local minimum of a differentiable function f. [Wikipedia]

- The gradient ∇f of a differentiable function f of several variables is a vector whose components are the partial derivatives of f. (simplified definition)
- The gradient of a function is the direction of steepest descent or ascent.
- Gradient descent means stepping in the opposite direction of the gradient.
- Likewise, gradient ascent means stepping in the direction of the gradient; it will lead to a local maximum of that function.

©STEIN 2005-2020

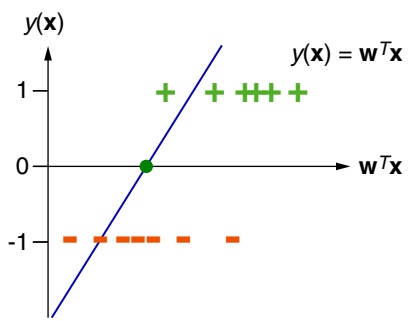
Remarks:

- Compared to the perceptron algorithm a gradient descent will converge even if X_1 and X_0 cannot be separated by a hyperplane. However, this convergence process is of an asymptotic nature and no finite iteration bound can be stated.

©STEIN 2005-2020

Gradient Descent

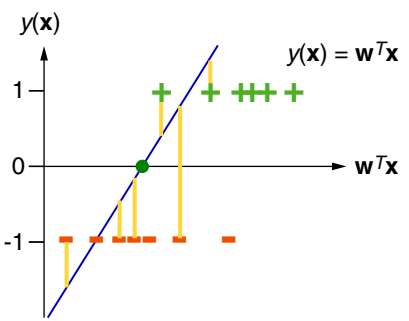
(1) Linear Regression + Squared Loss



©STEIN 2005-2020

Gradient Descent

(1) Linear Regression + Squared Loss (continued)



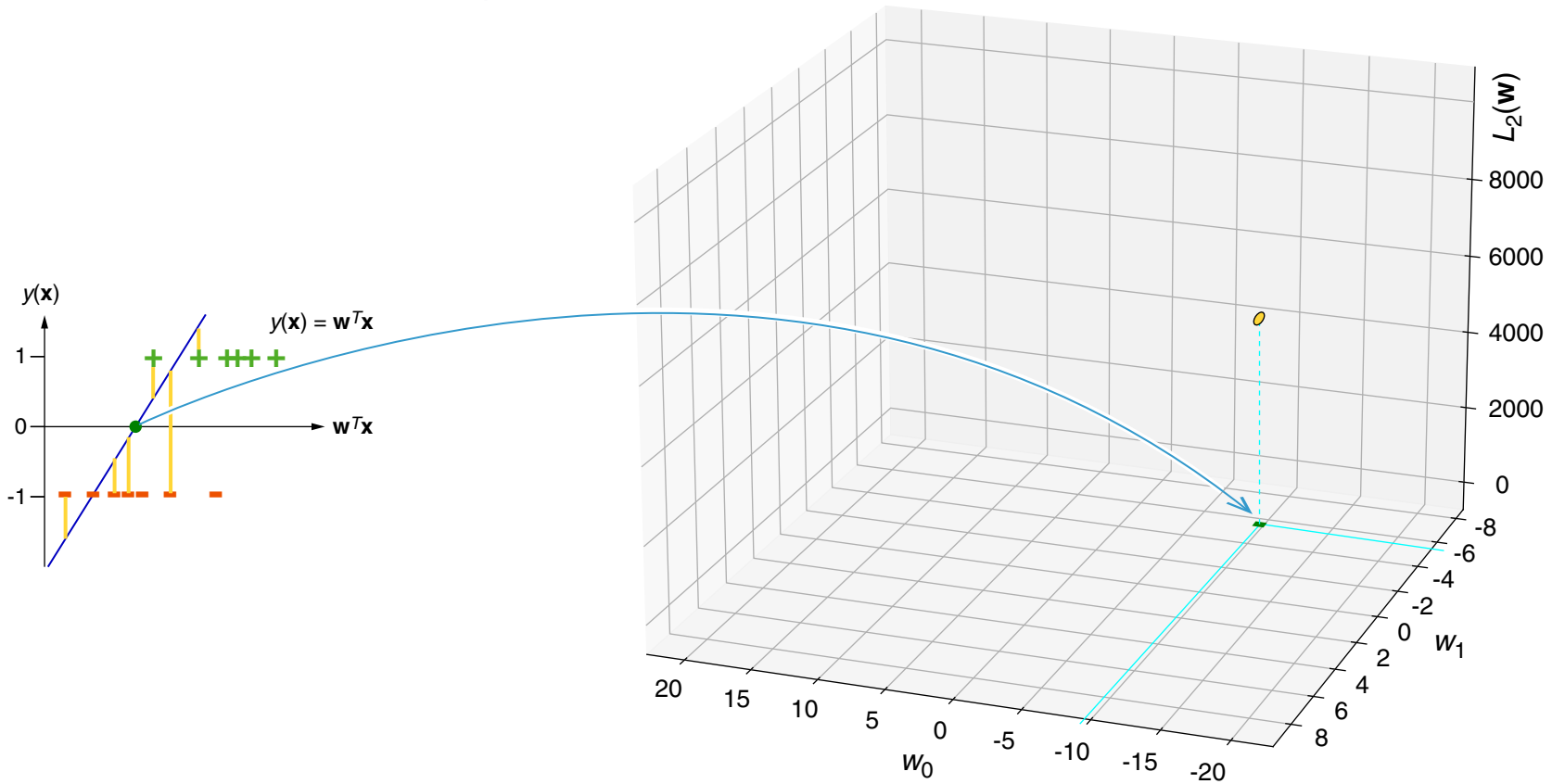
$$L_2(\mathbf{w}) \;=\; \frac{1}{2}\cdot \mathrm{RSS}(\mathbf{w}) \;=\; \frac{1}{2}\cdot \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \big(c(\mathbf{x}) - y(\mathbf{x})\big)^2$$

[Pointwise loss]

©STEIN 2005-2020

Gradient Descent

(1) Linear Regression + Squared Loss (continued)



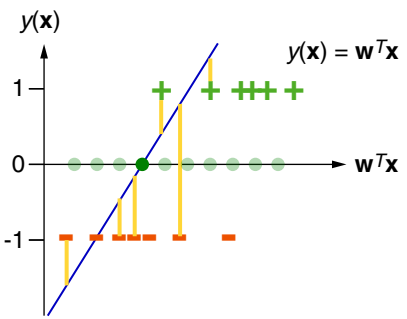
$$L_2(\mathbf{w}) \;=\; \frac{1}{2}\cdot \mathrm{RSS}(\mathbf{w}) \;=\; \frac{1}{2}\cdot \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \big(c(\mathbf{x}) - y(\mathbf{x})\big)^2$$

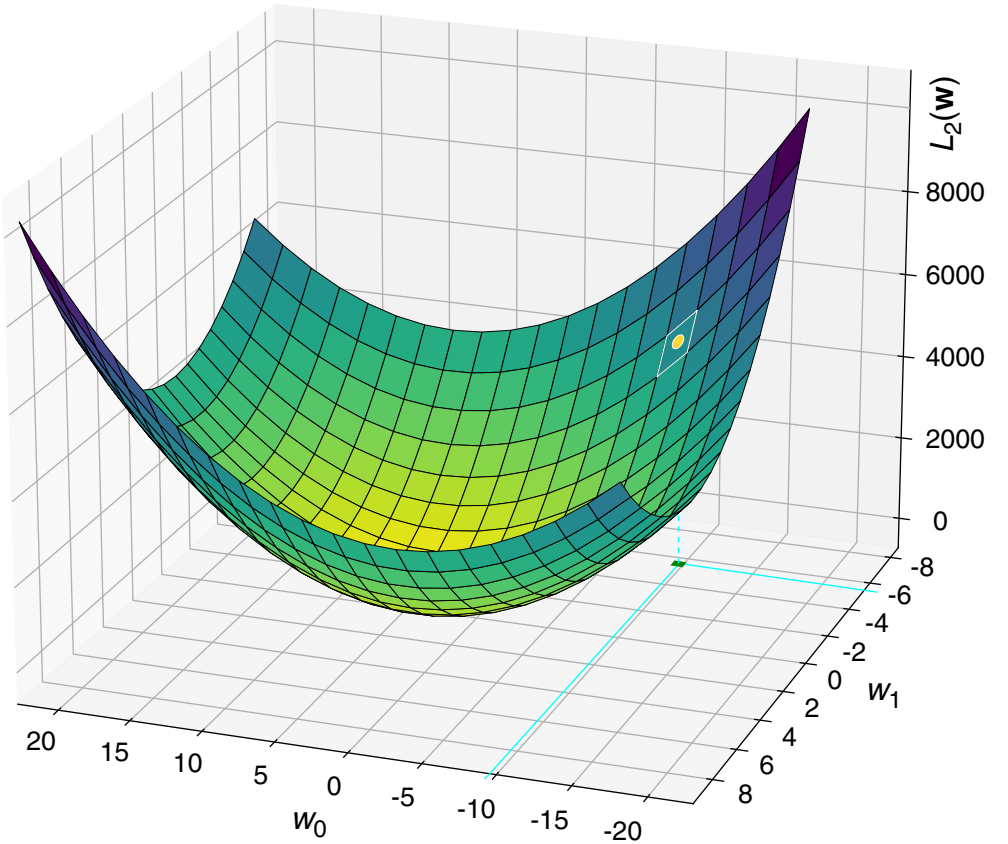
[Pointwise loss]

©STEIN 2005-2020

Gradient Descent

(1) Linear Regression + Squared Loss (continued)





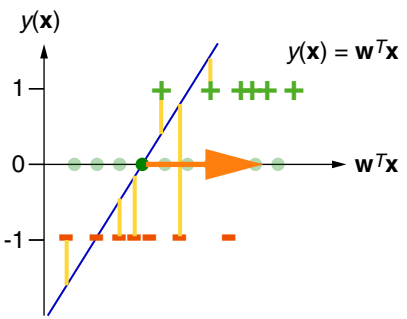
$$L_2(\mathbf{w}) \;=\; \frac{1}{2}\cdot \mathrm{RSS}(\mathbf{w}) \;=\; \frac{1}{2}\cdot \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \big(c(\mathbf{x}) - y(\mathbf{x})\big)^2$$

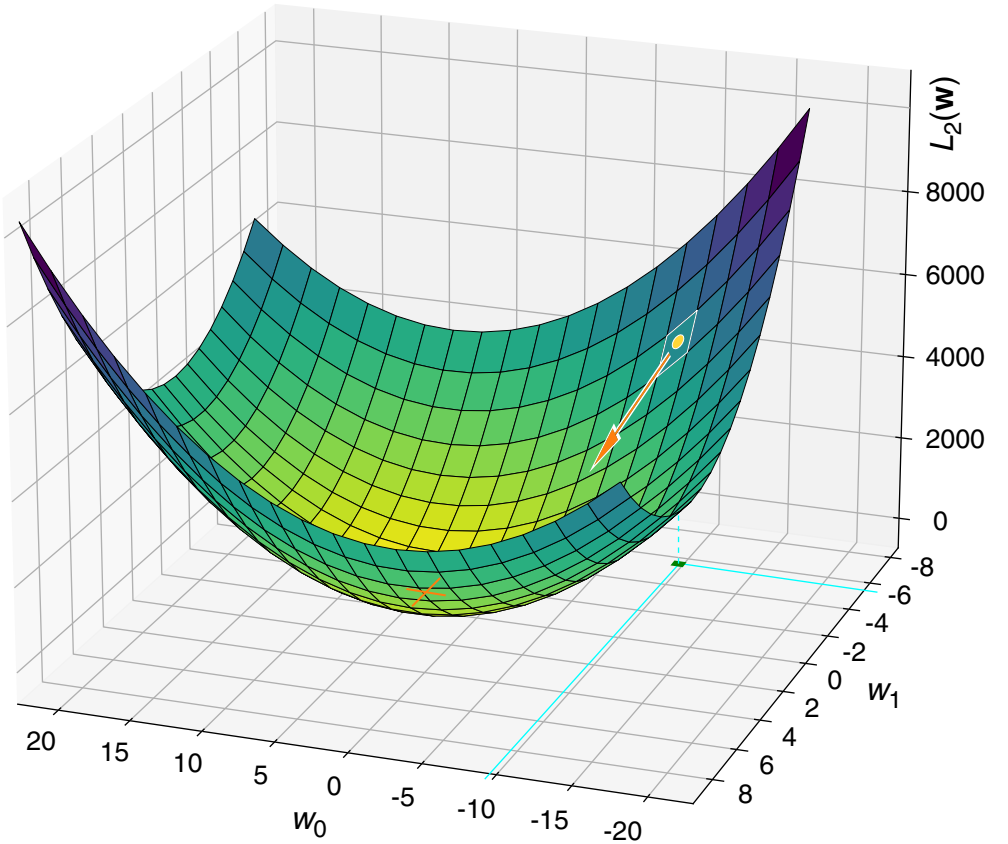
[Pointwise loss]

©STEIN 2005-2020

Gradient Descent

(1) Linear Regression + Squared Loss (continued)





$$L_2(\mathbf{w}) \;=\; \frac{1}{2}\cdot \mathrm{RSS}(\mathbf{w}) \;=\; \frac{1}{2}\cdot \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \big(c(\mathbf{x}) - y(\mathbf{x})\big)^2$$

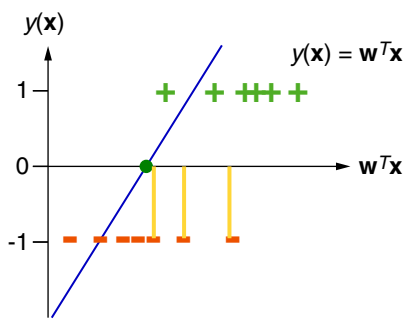
[Pointwise loss]

$$\nabla L_2(\mathbf{w}) \;=\; \left(\frac{\partial L_2(\mathbf{w})}{\partial w_0},\; \frac{\partial L_2(\mathbf{w})}{\partial w_1},\; \ldots,\; \frac{\partial L_2(\mathbf{w})}{\partial w_p}\right)$$

©STEIN 2005-2020

Gradient Descent

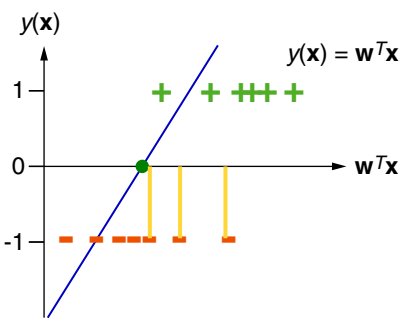
(2) Linear Regression + 0/1 Loss



©STEIN 2005-2020

Gradient Descent

(2) Linear Regression + 0/1 Loss (continued)

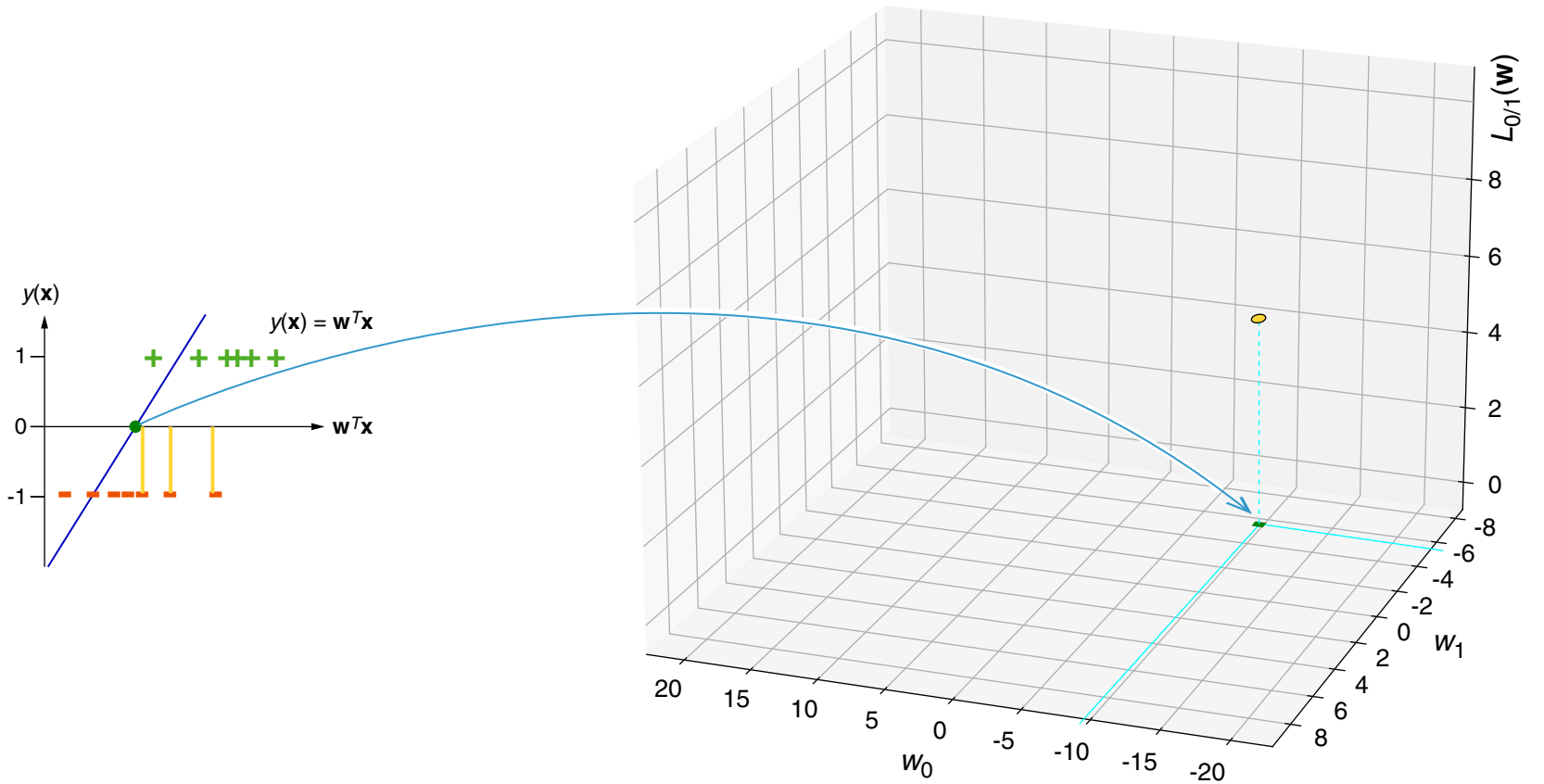


$$L_{0/1}(\mathbf{w}) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \frac{1}{2}\cdot(c(\mathbf{x}) - \text{sign}(\mathbf{w}^T\mathbf{x}))$$

©STEIN 2005-2020

Gradient Descent

(2) Linear Regression + 0/1 Loss (continued)

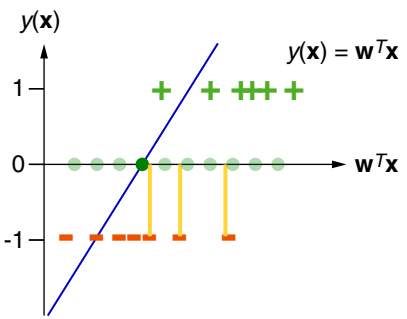


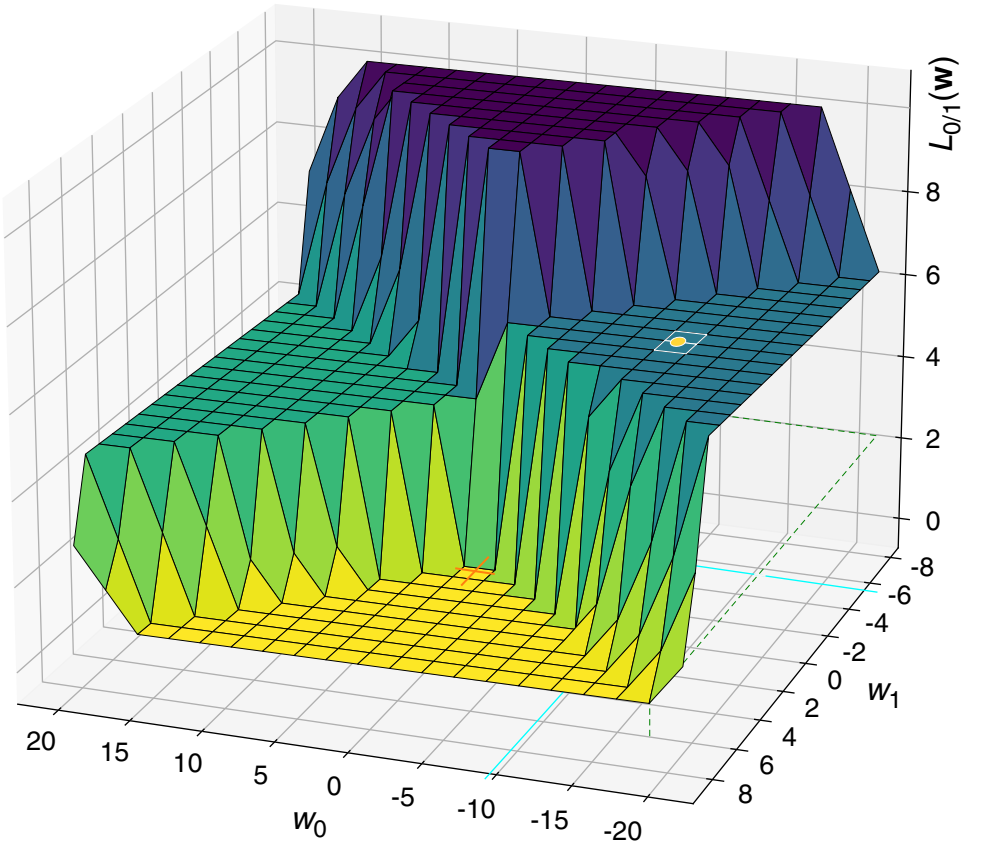
$$L_{0/1}(\mathbf{w}) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \frac{1}{2}\cdot(c(\mathbf{x}) - \text{sign}(\mathbf{w}^T\mathbf{x}))$$

©STEIN 2005-2020

Gradient Descent

(2) Linear Regression + 0/1 Loss (continued)





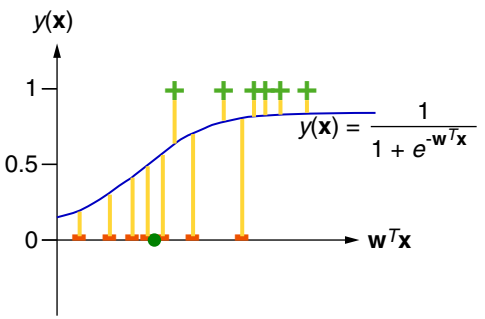
$$L_{0/1}(\mathbf{w}) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} I(c(\mathbf{x}), \text{sign}(y(\mathbf{x}))) \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \frac{1}{2}\cdot(c(\mathbf{x}) - \text{sign}(\mathbf{w}^T\mathbf{x}))$$

$\nabla L_{0/1}(\mathbf{w})$ cannot be expressed as a differentiable function.

©STEIN 2005-2020

Gradient Descent

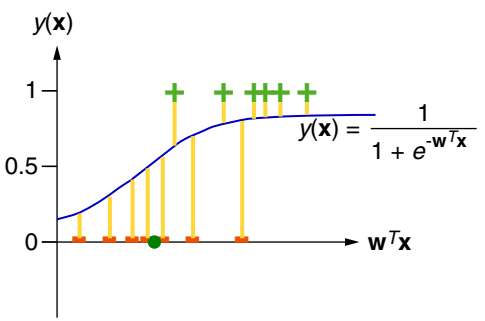
(3) Logistic Regression + Logistic Loss + Regularization



©STEIN 2005-2020

Gradient Descent

(3) Logistic Regression + Logistic Loss + Regularization (continued)



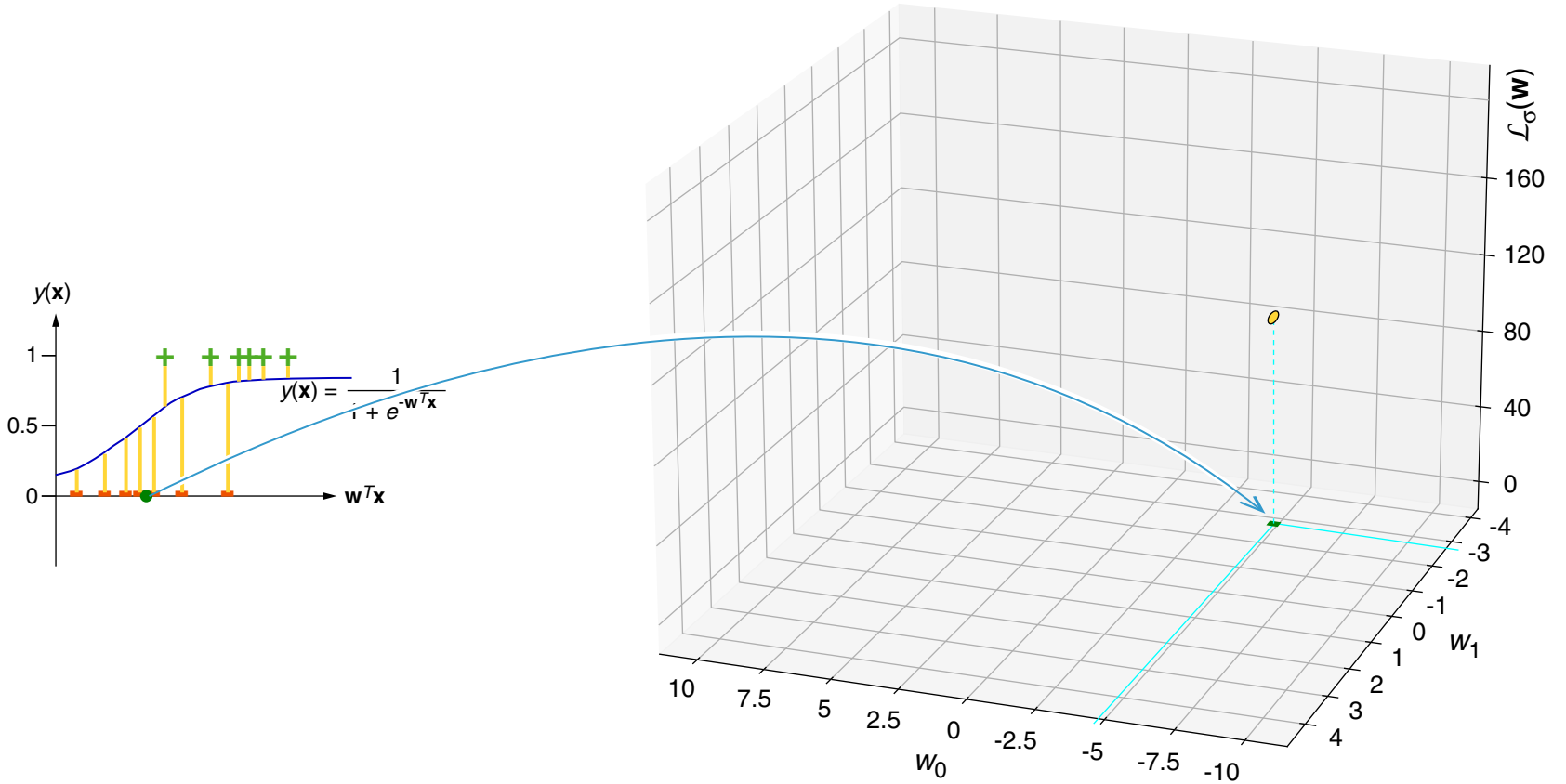
$$\mathcal{L}_\sigma(\mathbf{w}) \;=\; L_\sigma + R_{\|w\|^2} \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

$$= \sum_{(\mathbf{x},c(\mathbf{x}))\in D} -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

©STEIN 2005-2020

Gradient Descent

(3) Logistic Regression + Logistic Loss + Regularization (continued)



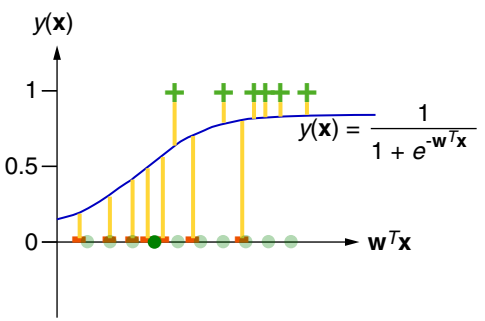
$$\mathcal{L}_\sigma(\mathbf{w}) \;=\; L_\sigma + R_{\|w\|^2} \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

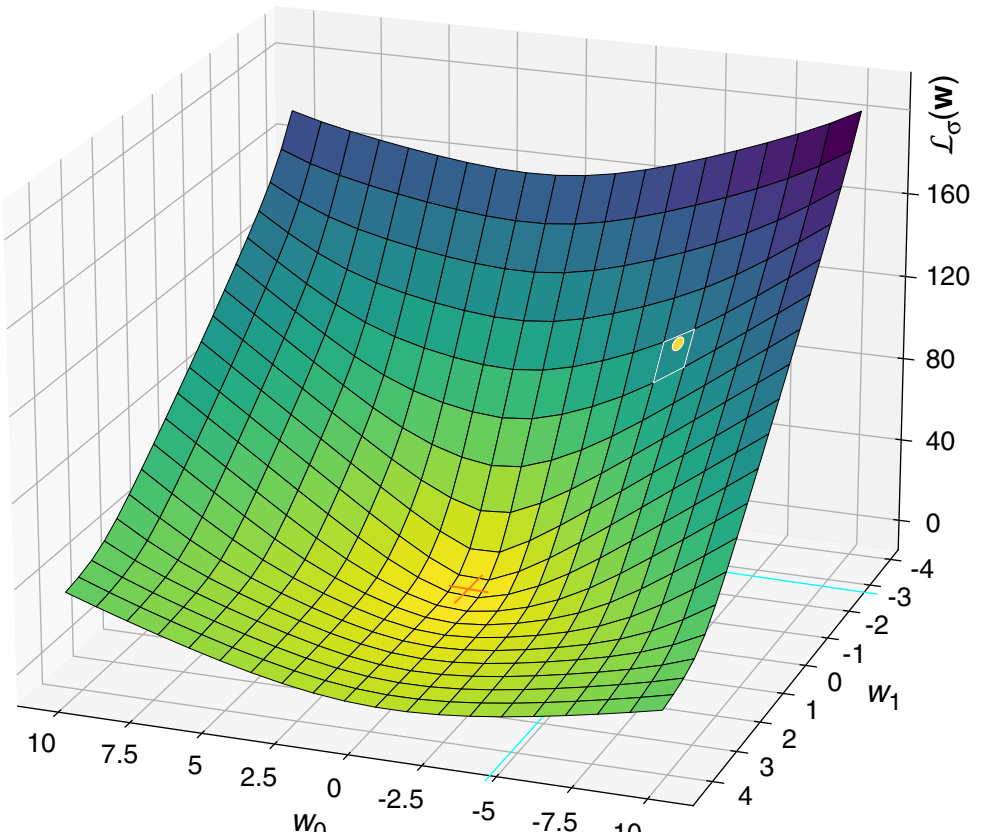
$$= \sum_{(\mathbf{x},c(\mathbf{x}))\in D} -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

©STEIN 2005-2020

Gradient Descent

(3) Logistic Regression + Logistic Loss + Regularization (continued)





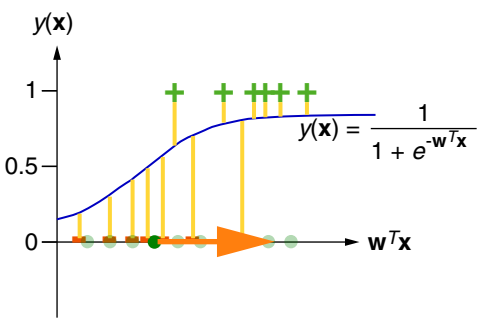
$$\mathcal{L}_\sigma(\mathbf{w}) \;=\; L_\sigma + R_{\|w\|^2} \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

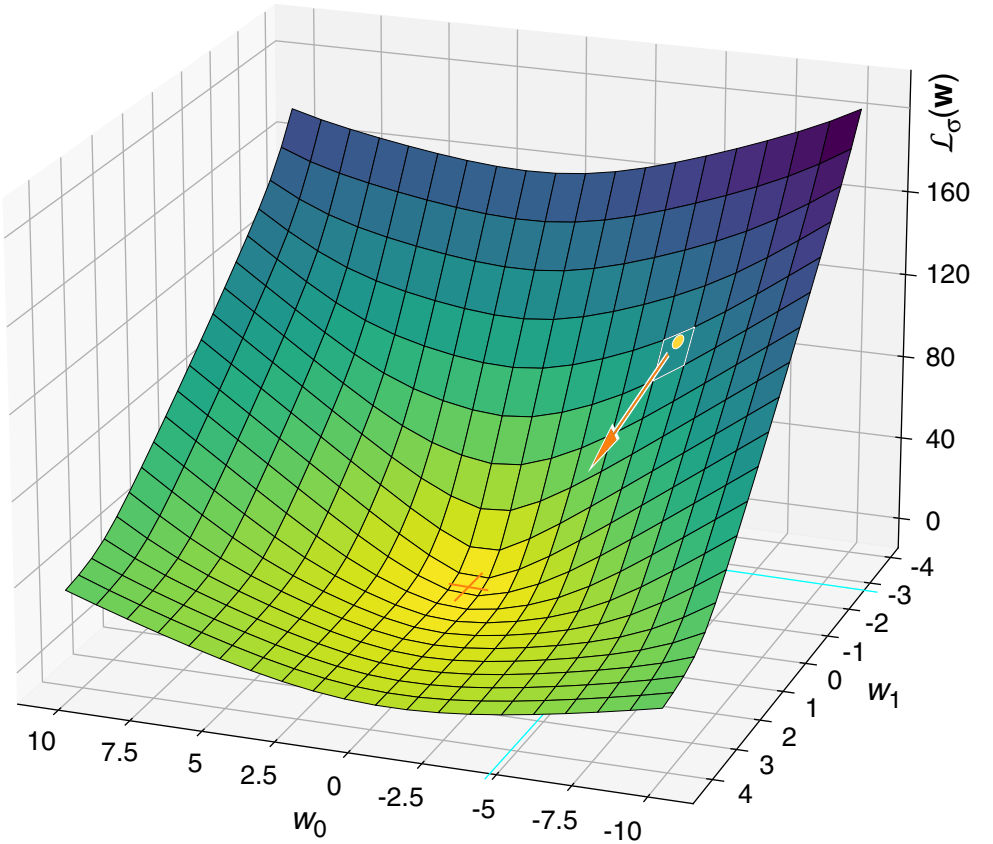
$$= \sum_{(\mathbf{x},c(\mathbf{x}))\in D} -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1-c(\mathbf{x})) \cdot \log(1-y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

©STEIN 2005-2020

Gradient Descent

(3) Logistic Regression + Logistic Loss + Regularization (continued)





$$\mathcal{L}_\sigma(\mathbf{w}) \;=\; L_\sigma + R_{\|w\|^2} \;=\; \sum_{(\mathbf{x},c(\mathbf{x}))\in D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \cdot \mathbf{w}^T\mathbf{w}$$

$$\nabla\mathcal{L}_\sigma(\mathbf{w}) \;=\; \left(\frac{\partial\mathcal{L}_\sigma(\mathbf{w})}{\partial w_0},\; \frac{\partial\mathcal{L}_\sigma(\mathbf{w})}{\partial w_1},\; \ldots,\; \frac{\partial\mathcal{L}_\sigma(\mathbf{w})}{\partial w_p}\right)$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w})$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x})$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w})$$

$$\frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x})$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T \mathbf{x})(-x_j)$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Linear Regression + Squared Loss

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla L_2(\mathbf{w})$$ [*PT* Algorithm]

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j = -\eta \frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \eta \sum_{(\mathbf{x},\, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot x_j$$

$$\frac{\partial}{\partial w_j} L_2(\mathbf{w}) = \frac{\partial}{\partial w_j} \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - y(\mathbf{x}))^2 = \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))^2$$

$$= \frac{1}{2} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} 2(c(\mathbf{x}) - y(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - y(\mathbf{x}))$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot \frac{\partial}{\partial w_j} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})$$

$$= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})(-x_j)$$

©STEIN 2005-2020

Gradient Descent

The BGD Algorithm

[Algorithms: *LMS* *BGD*$_\sigma$ *PT* *BGD*$_2$ *IGD*]

Algorithm:	*BGD*	Batch Gradient Descent
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\lvert\mathbf{x}\rvert = p+1,\ c(\mathbf{x}) \in \{0,1\}$. // $c(\mathbf{x}) \in \{-1,1\}$
	η	Learning rate, a small positive constant.
Internal:	$y(D)$	Set of $y(\mathbf{x})$-values computed from the elements $\mathbf{x}$ in D given some $\mathbf{w}$.
Output:	$\mathbf{w}$	Weight vector.

BGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
   t = t + 1
   Δw = 0
   FOREACH (x, c(x)) ∈ D DO
      δ = c(x) − w^T x
      Δw = Δw + η · δ · x
   ENDDO
   w = w + Δw
UNTIL(convergence(D, y(D)) OR t > t_max)
return(w)
```


©STEIN 2005-2020

Remarks:

- Recap. Each BGD iteration "`REPEAT ... UNTIL`" (1) computes the direction of steepest loss descent as $-\nabla L_2(\mathbf{w}_t) = \sum_{(\mathbf{x},c(\mathbf{x}))\in D} (c(\mathbf{x}) - y_t(\mathbf{x})) \cdot \mathbf{x}$ with $y_t(\mathbf{x}) = \mathbf{w}_t^T\mathbf{x}$, and (2) updates $\mathbf{w}_t$ by taking a step of length η in this direction.

- Recap. The *convergence*() function may analyze the global logistic loss, $L_2(\mathbf{w}_t)$, or the norm of the loss gradient, $||\nabla L_2(\mathbf{w_t})||$, and compare it to some small positive bound ε.

- The speed of convergence of the basic gradient descent may be considered unsatisfactory. Even when taking the optimal step size η at each iteration, it can be shown that gradient descent has merely a linear rate of convergence. [Meza 2010]

  More advanced numerical algorithms to tackle the optimization (search for minimum L_σ) are listed below but are not treated in detail here:

  - Newton-Raphson algorithm
  - Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm
  - Limited-memory BFGS (L-BFGS)
  - Conjugate gradient method

©STEIN 2005-2020

Gradient Descent

Global Loss versus Pointwise Loss

The weight adaptation in the *BGD* Algorithm happens "set-based": for adapting a component of $\mathbf{w}$, the global loss of all examples (the "batch") is computed.

Weight adaptation with regard to a *single* example $(\mathbf{x}, c(\mathbf{x})) \in D$:

$$\Delta\mathbf{w} \;=\; \eta \cdot (c(\mathbf{x}) - \mathbf{w}^T\mathbf{x}) \cdot \mathbf{x}$$

This adaptation rule is known under different names:

- delta rule
- Widrow-Hoff rule
- adaline rule
- least mean squares (LMS) rule

The loss with regard to a single example $(\mathbf{x}, c(\mathbf{x})) \in D$, also called pointwise loss is given as:

$$\ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) \;=\; \frac{1}{2}\,(c(\mathbf{x}) - \mathbf{w}^T\mathbf{x})^2 \qquad \text{[Global loss]}$$

©STEIN 2005-2020

Gradient Descent

The IGD Algorithm

[Algorithms: *LMS* *BGD*$_\sigma$ *PT* *BGD*$_2$ *IGD*]

Algorithm:	*IGD*	Incremental Gradient Descent
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\lvert\mathbf{x}\rvert = p+1,\ c(\mathbf{x}) \in \{0,1\}$. // $c(\mathbf{x}) \in \{-1,1\}$
	η	Learning rate, a small positive constant.
Internal:	$y(D)$	Set of $y(\mathbf{x})$-values computed from the elements $\mathbf{x}$ in D given some $\mathbf{w}$.
Output:	$\mathbf{w}$	Weight vector.

IGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
  t = t + 1
  FOREACH (x, c(x)) ∈ D DO
    error = c(x) − w^T x
    Δw = η · error · x
    w = w + Δw
  ENDDO
UNTIL(convergence(D, y(D)) OR t > t_max)
return(w)
```


©STEIN 2005-2020

Remarks:

- The sequence of incremental weight adaptations approximates the gradient descent of the batch approach. If η is chosen sufficiently small, this approximation can happen at arbitrary accuracy.
- The computation of the total error of batch gradient descent enables larger weight adaptation increments.
- Compared to batch gradient descent, the example-based weight adaptation of incremental gradient descent can better avoid getting stuck in a local minimum of the error function.
- A related method is *stochastic* gradient descent, which estimates the gradient from a randomly selected subset of the data.

©STEIN 2005-2020

Remarks (continued):

- When, as is done here, the residual sum of squares, RSS, is chosen as error (loss) function, the incremental gradient descent algorithm [*IGD*] is very similar to the least mean squares algorithm [*LMS*].

- The incremental gradient descent algorithm [*IGD*] looks similar to the perceptron training algorithm [*PT*], since these algorithms differ only in the error computation (Line 5) where the latter applies the Heaviside function. However, this subtle syntactic difference is a significant conceptual difference, entailing a number of consequences:
  - Gradient descent is a regression approach and exploits the residua, which are provided by an error function of choice, and whose differential is evaluated to control the hyperplane movement.
  - The *PT* algorithm is not based on residuals (in the $(p+1)$-dimensional input-output-space) but refers to the input space only, where it simply evaluates the side of the hyperplane as a binary feature (correct side or not).
  - Provided linear separability, the *PT* algorithm will converge within a finite number of iterations, which, however, cannot be guaranteed for gradient descent.
  - Gradient descent may converge even if the data is not linearly separable.
  - Data sets can be constructed whose classes are linearly separable, but where gradient descent will not determine a hyperplane that classifies all examples correctly (whereas the *PT* Algorithm of course does).

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Logistic Regression + Logistic Loss + Regularization

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathcal{L}_\sigma(\mathbf{w})$$

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$\begin{aligned} w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j &= -\eta \frac{\partial}{\partial w_j} \mathcal{L}_\sigma(\mathbf{w}) \\ &= \eta \sum_{(\mathbf{x},\, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \sigma(\mathbf{w}^T \mathbf{x})) \cdot x_j - \eta \cdot 2\lambda \cdot w_j \end{aligned}$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Logistic Regression + Logistic Loss + Regularization

$$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w} \quad \text{where} \quad \Delta\mathbf{w} = -\eta \nabla \mathcal{L}_\sigma(\mathbf{w})$$

Componentwise ($j = 0, \ldots, p$) weight adaptation:

$$\begin{aligned} w_j \leftarrow w_j + \Delta w_j \quad \text{where} \quad \Delta w_j &= -\eta \frac{\partial}{\partial w_j} \mathcal{L}_\sigma(\mathbf{w}) \\ &= \eta \sum_{(\mathbf{x},\, c(\mathbf{x})) \in D} (c(\mathbf{x}) - \sigma(\mathbf{w}^T\mathbf{x})) \cdot x_j - \eta \cdot 2\lambda \cdot w_j \end{aligned}$$

$$\begin{aligned} \frac{\partial}{\partial w_j} \mathcal{L}_\sigma(\mathbf{w}) &= \frac{\partial}{\partial w_j} L_\sigma(\mathbf{w}) + R_{\|w\|^2}(\mathbf{w}) = \frac{\partial}{\partial w_j} \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \mathbf{w}^T\mathbf{w} \\ &= \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \frac{\partial}{\partial w_j} \ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) + \lambda \frac{\partial}{\partial w_j} \mathbf{w}^T\mathbf{w} \end{aligned}$$

©STEIN 2005-2020

Gradient Descent

Weight Adaptation: Logistic Regression + Logistic Loss + Regularization

Details for the two summands:

$$
\begin{aligned}
\frac{\partial}{\partial w_j}\ell_\sigma(c(\mathbf{x}), y(\mathbf{x})) &= \frac{\partial}{\partial w_j} - c(\mathbf{x}) \cdot \mathsf{log}(\sigma(\mathbf{w}^T\mathbf{x})) - (1 - c(\mathbf{x})) \cdot \mathsf{log}(1 - \sigma(\mathbf{w}^T\mathbf{x})) \\
&= -c(\mathbf{x}) \cdot \frac{\partial}{\partial w_j} \mathsf{log}(\sigma(\mathbf{w}^T\mathbf{x})) - (1 - c(\mathbf{x})) \cdot \frac{\partial}{\partial w_j} \mathsf{log}(1 - \sigma(\mathbf{w}^T\mathbf{x})) \\
&= -c(\mathbf{x})\frac{1}{\sigma(\mathbf{w}^T\mathbf{x})}\frac{\partial}{\partial w_j}\sigma(\mathbf{w}^T\mathbf{x}) - (1 - c(\mathbf{x}))\frac{1}{1 - \sigma(\mathbf{w}^T\mathbf{x})}(-\frac{\partial}{\partial w_j}\sigma(\mathbf{w}^T\mathbf{x})) \\
&= -c(\mathbf{x})\frac{1}{\sigma(\mathbf{w}^T\mathbf{x})}\sigma(\mathbf{w}^T\mathbf{x})(1 - \sigma(\mathbf{w}^T\mathbf{x}))\frac{\partial}{\partial w_j}\mathbf{w}^T\mathbf{x} \\
&\quad -(1 - c(\mathbf{x}))\frac{1}{1 - \sigma(\mathbf{w}^T\mathbf{x})}(-1)\sigma(\mathbf{w}^T\mathbf{x})(1 - \sigma(\mathbf{w}^T\mathbf{x}))\frac{\partial}{\partial w_j}\mathbf{w}^T\mathbf{x} \\
&= -c(\mathbf{x})(1 - \sigma(\mathbf{w}^T\mathbf{x})) \cdot x_j + (1 - c(\mathbf{x}))\sigma(\mathbf{w}^T\mathbf{x}) \cdot x_j \\
&= -(c(\mathbf{x}) - \sigma(\mathbf{w}^T\mathbf{x})) \cdot x_j
\end{aligned}
$$

$$
\frac{\partial}{\partial w_j}\mathbf{w}^T\mathbf{w} = \frac{\partial}{\partial w_j}\sum_{i=1}^{p} w_i^2 = \frac{\partial}{\partial w_j} w_j^2 = 2w_j
$$

©STEIN 2005-2020